RELATIONAL DATABASE VS. NoSQL DATABASE

ASSIGNMENT 4

DATABASE MANAGEMENT SYSTEM

ALLIANCE UNIVERSITY

Department of Computer Applications

SUMMITED BY: BHUVANA S

REGISTER NO: 2511021240036

COURSE: BCA

YEAR: 2ND YEAR

Introduction

Database technology is essential for managing the data generated by real-world applications. Relational and NoSQL databases are two widely used database technologies, each offering different features for storing, organizing, and retrieving information. Relational databases use tables and relationships to maintain structured data, whereas NoSQL databases support flexible data models to meet different application requirements. This assignment compares relational and NoSQL databases in terms of scalability, consistency, performance, and use cases. It also examines why organizations select different database technologies based on their needs, with reference to the Food Delivery Management System, where relational databases can manage customer details, restaurant information, orders, payments, and delivery records efficiently.

Comparison Between Relational Databases and NoSQL Databases

A relational database stores data in tables consisting of rows and columns. It uses SQL (Structured Query Language) to create, retrieve, update, and delete data. Examples include MySQL, PostgreSQL, and Oracle Database.

A NoSQL database stores data using models such as documents, key-value pairs, columns, or graphs. It is designed to support different data structures and flexible data storage. Examples include MongoDB, Redis, and Apache Cassandra.

The following table compares relational databases and NoSQL databases.

Feature	**Relational Database**	**NoSQL Database**
Data Structure	Stores data in tables with rows and columns.	Stores data as documents, key-value pairs, columns, or graphs.
Scalability	Commonly scales vertically by increasing the resources of a server; horizontal scaling is also possible.	Often designed to support horizontal scaling across multiple servers.
Consistency	Typically provides strong consistency and supports ACID transactions.	Consistency depends on the database; some support strong consistency, while others offer eventual consistency.
Performance	Performs well for structured data, complex queries, and relationships between tables.	Can perform well for large volumes of data and workloads suited to its data model.
Schema	Usually requires a predefined schema.	Often supports flexible schemas, depending on the database model.
Relationships	Uses primary keys, foreign keys, and joins to connect tables.	Relationships may be represented using embedded documents, references, or

		other model-specific structures.
Data Integrity	Supports constraints such as PRIMARY KEY, FOREIGN KEY, UNIQUE, and CHECK.	Data integrity features depend on the database and its configuration.
Suitable Use Cases	Banking, college management, inventory management, and order processing.	Real-time analytics, content management, caching, and large-scale distributed applications.
Examples	MySQL, PostgreSQL, Oracle Database.	MongoDB, Redis, Apache Cassandra.

Why Organizations Choose Different Database Technologies

Organizations choose relational or NoSQL databases based on their application requirements, data structure, consistency needs, performance expectations, and scalability requirements.

Relational Databases:

Organizations prefer relational databases when data is structured and relationships between records must be maintained accurately.

Reasons for choosing relational databases include:

1. **Structured data:** Data can be organized into tables with predefined columns and data types.
2. **Data integrity:** Primary keys, foreign keys, and other constraints help maintain accurate and consistent records.
3. **Transaction support:** ACID transactions help ensure reliable operations, especially when multiple related records must be updated together.
4. **Complex queries:** SQL supports filtering, grouping, aggregation, and joins across multiple tables.
5. **Suitable applications:** Relational databases are useful for banking, financial systems, inventory management, and order processing.

Example: A bank may use a relational database to manage customer accounts and transactions because financial operations require reliable transaction processing and consistency.

NoSQL Databases:

Organizations may prefer NoSQL databases when they need flexible data structures, horizontal scalability, or a database suited to large-scale distributed workloads.

Reasons for choosing NoSQL databases include:

1. **Flexible data models:** Some NoSQL databases allow records to contain different fields without requiring the same fixed structure for every record.
2. **Horizontal scalability:** Many NoSQL systems are designed to distribute data across multiple servers.
3. **Large-scale workloads:** Some NoSQL databases are suitable for applications that process large amounts of data across distributed systems.
4. **Application-specific performance:** Key-value stores, document databases, and graph databases can be effective for particular access patterns.
5. **Suitable applications:** NoSQL databases are useful for caching, content management, real-time applications, and distributed data storage.

Example: An application may use MongoDB to store product catalogues containing different attributes for different product types, such as clothing, electronics, and furniture.

Comparative Report with Real-World Examples

Relational and NoSQL databases serve different purposes, and the appropriate choice depends on the requirements of the application.

Application	Suitable Database Type	Example	Reason
Banking System	Relational	PostgreSQL	Supports structured financial records, transactions, and integrity constraints.
College Management System	Relational	MySQL	Manages students, courses, attendance, and examination records through related tables.
Food Delivery Management System	Relational	MySQL	Connects customers, restaurants, menu items, orders, and payments through keys and relationships.
Product Catalogue	NoSQL	MongoDB	Supports flexible product documents with different attributes.

Distributed Caching	NoSQL	Redis	Provides fast access to data stored as key-value entries.
Large Distributed Data Workloads	NoSQL	Apache Cassandra	Supports distributed storage and horizontal scalability.

Application to the Food Delivery Management System

For the Food Delivery Management System developed in the previous assignments, a relational database such as MySQL is a suitable choice.

The application contains seven related tables: CUSTOMER, RESTAURANT, MENU_ITEM, ORDERS, ORDER_ITEM, PAYMENT, and DELIVERY_PARTNER.

A relational database is appropriate because:

- Each table has a defined structure and specific attributes.
- Primary keys uniquely identify records.
- Foreign keys establish relationships between customers, restaurants, orders, and other records.
- SQL joins can generate reports involving multiple tables.
- Constraints help maintain data integrity.
- Transactions can help ensure that related order and payment operations are handled reliably.

A NoSQL database could also be considered if the application later required flexible product data, large-scale distributed workloads, or other features suited to a particular NoSQL model. However, the relational model is a practical choice for the current database design.

Conclusion

Relational and NoSQL databases have different strengths and are selected according to the needs of an application. Relational databases are suitable for structured data, relationships, integrity constraints, and transactional operations. NoSQL databases offer flexible data models and can support horizontally scaled, distributed workloads. Organizations should evaluate their data structure, consistency requirements, performance needs, and scalability goals before selecting a database technology. For the Food Delivery Management System, a relational database is suitable because it manages structured information and maintains relationships between the seven tables.